# Transformer

Część pierwsza ćwiczeń z transformers

## Po co nowy mechanizm?

- W tradycyjnych RNN/LSTM informacja przepływa sekwencyjnie. Mechanizmy uwagi natomiast pozwalają modelowi skupić się na różnych częściach wejścia przy generowaniu każdego elementu wyjścia.
- W Transformerach uwaga jest podstawowym mechanizmem: każdy token oblicza "ważność" innych tokenów przez iloczyn skalarny swoich zapytań (queries) z kluczami (keys) i wykorzystuje wartości (values) do zbudowania reprezentacji.

## Attention is All You Need

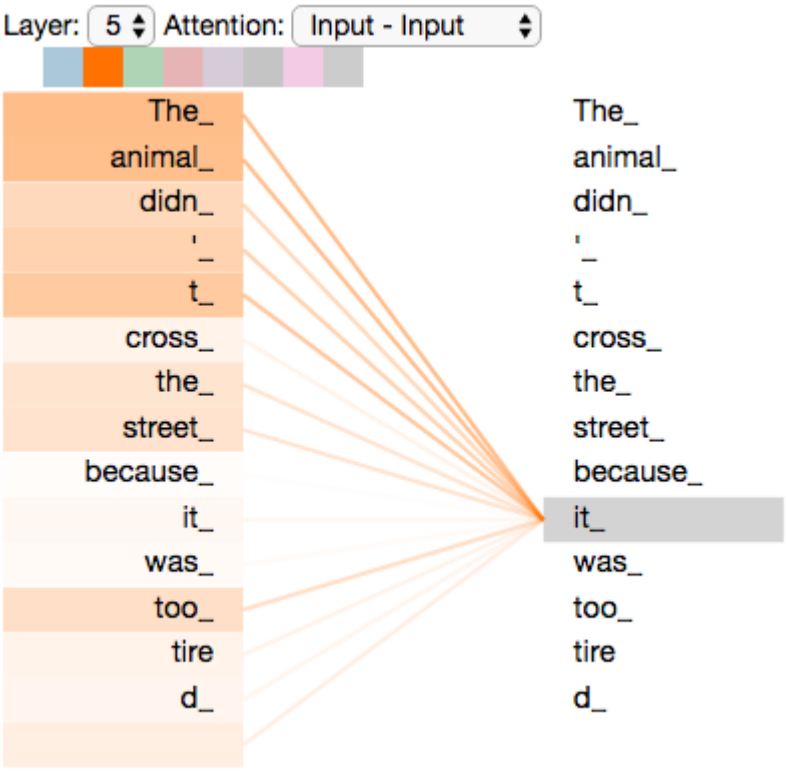
Mechanizm, który odpowiada za dobrą reputację transformerów to mechanizm attencji / uwagi. Mechanizm ten został zaprezentowany w research paperze: Attention is All You Need

### 1 Wejście

Na wejściu mamy takie zdanie:

```
”The animal didn't cross the street because it was too
tired”
```

W tym zdaniu modele językowe moga mieć problem zrozumieć do czego odnosi się słowo 'it".




## 2 Embeddingi

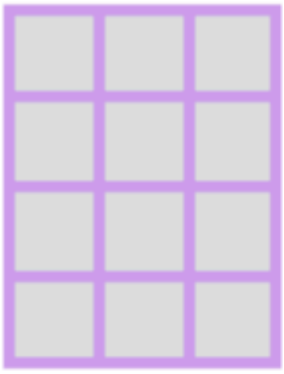
Przyszły do nas dwa słowa, które mają swoje odpowiadające embeddingi - x1 i x2



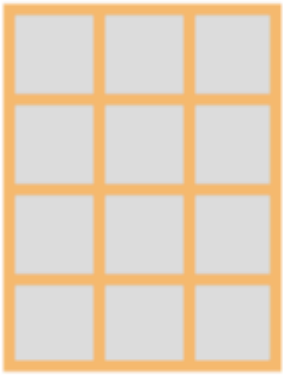

## 3 Query, key i value

- Q: query / zapytania — reprezentacja tego, czego "szukamy".
- K: key / klucze — reprezentacje elementów, do których porównujemy zapytania.
- V: values / wartości — zawierają informacje, które będziemy agregować (przez ważoną sumę).

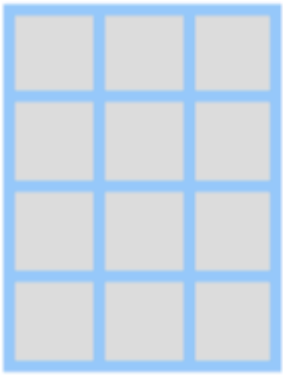
Podczas treningu modelu tworzymy 3 macierze - wagi dla wartości query, key i



$W^Q$



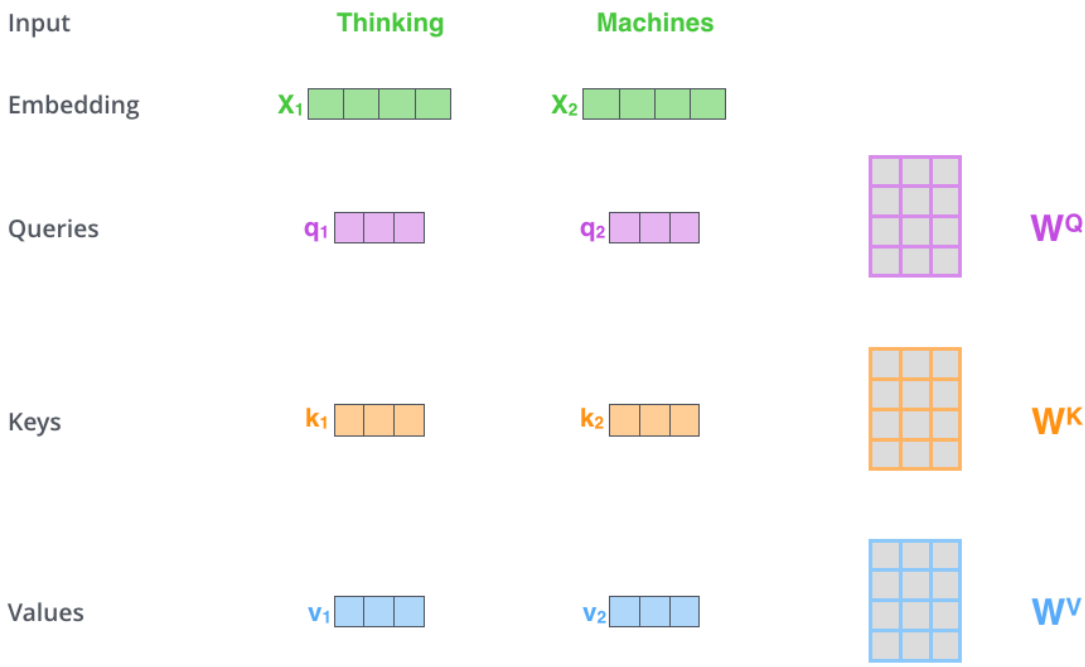
$W^K$



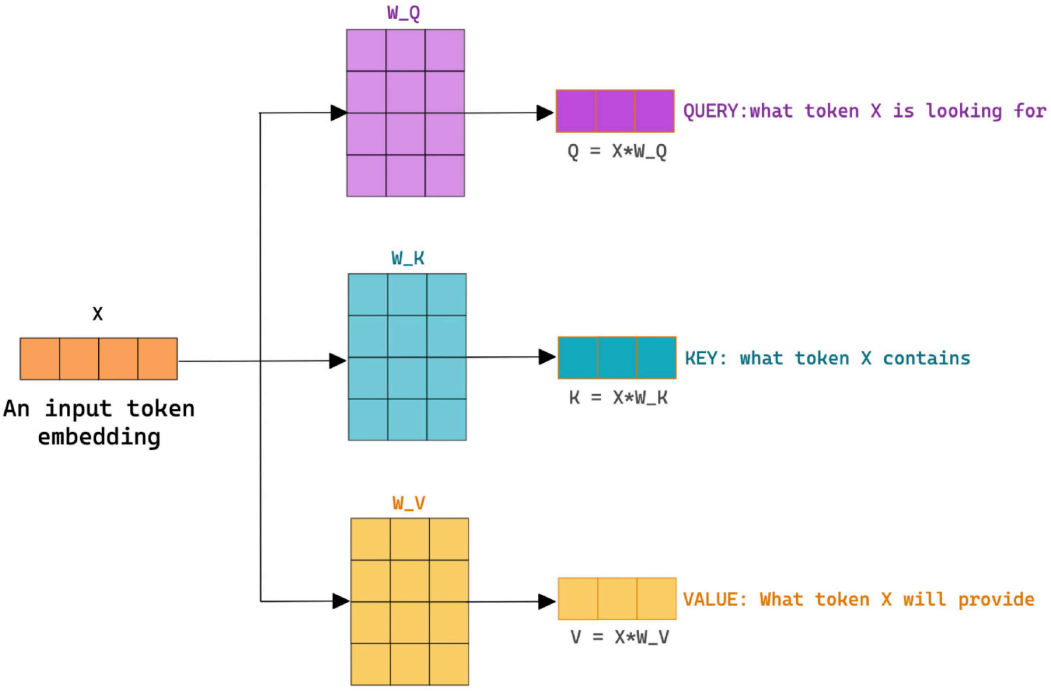
$W^V$

value:

Każde ze słów jest przemnożone przez te macierze:



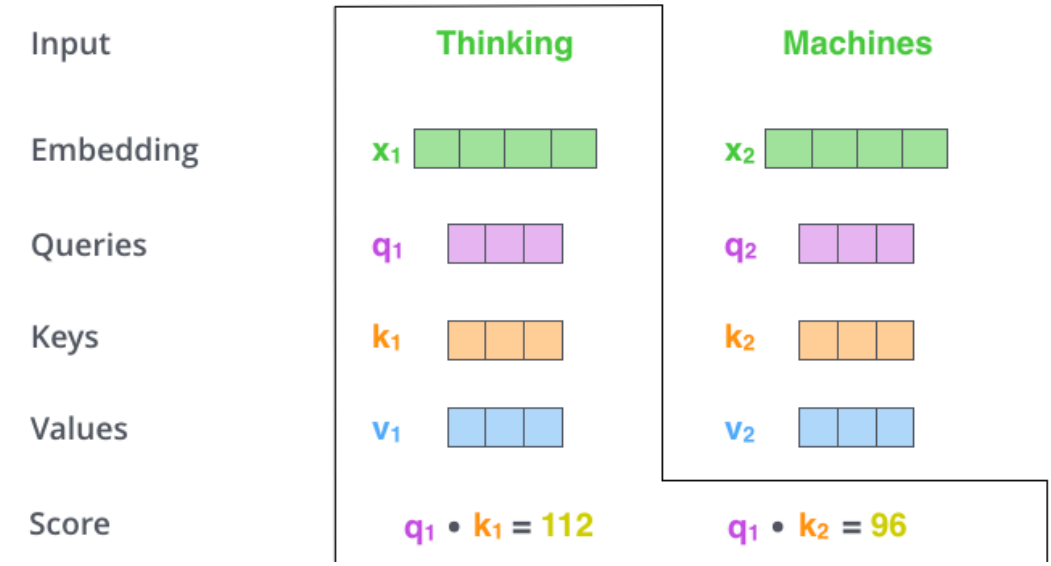

W ten sposób dla każdego słowa (embedingu) otrzymaliśmy wektory query, key i value. Raczej nie tłumaczymy tych słów na język polski.




## 4 Obliczenie attention - na wektorach

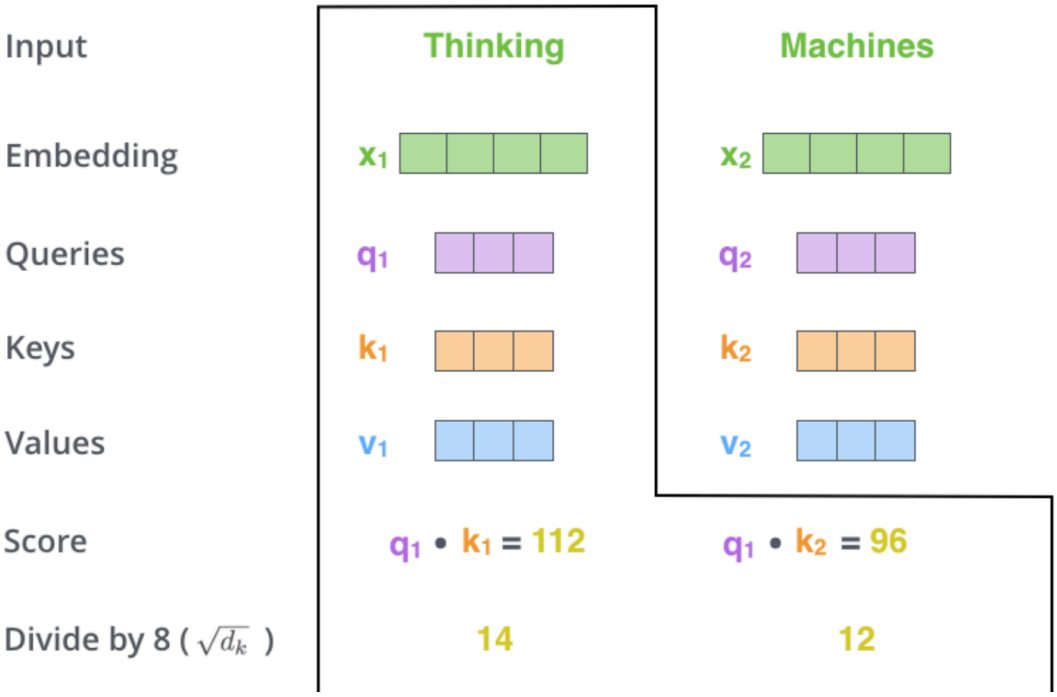
Obliczamy attention dla pierwszego słowa. Interesuje nas wynik jaki uzyskuje nasze z słowo z KAŻDYM słowem występującym.

Przemnażamy query (tego słowa) * key (po koeli każdego występującego słowa):

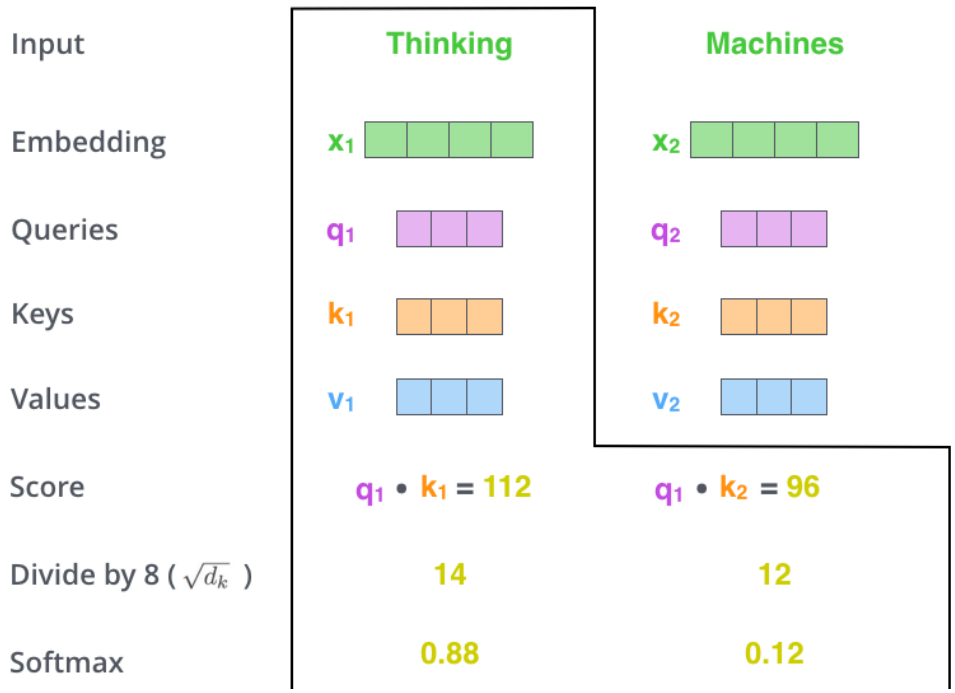



Dzielimy przez pierwiastek z wymiaru wektoru k:

(na podanym przykładzie wzięto wymiar k = 64)

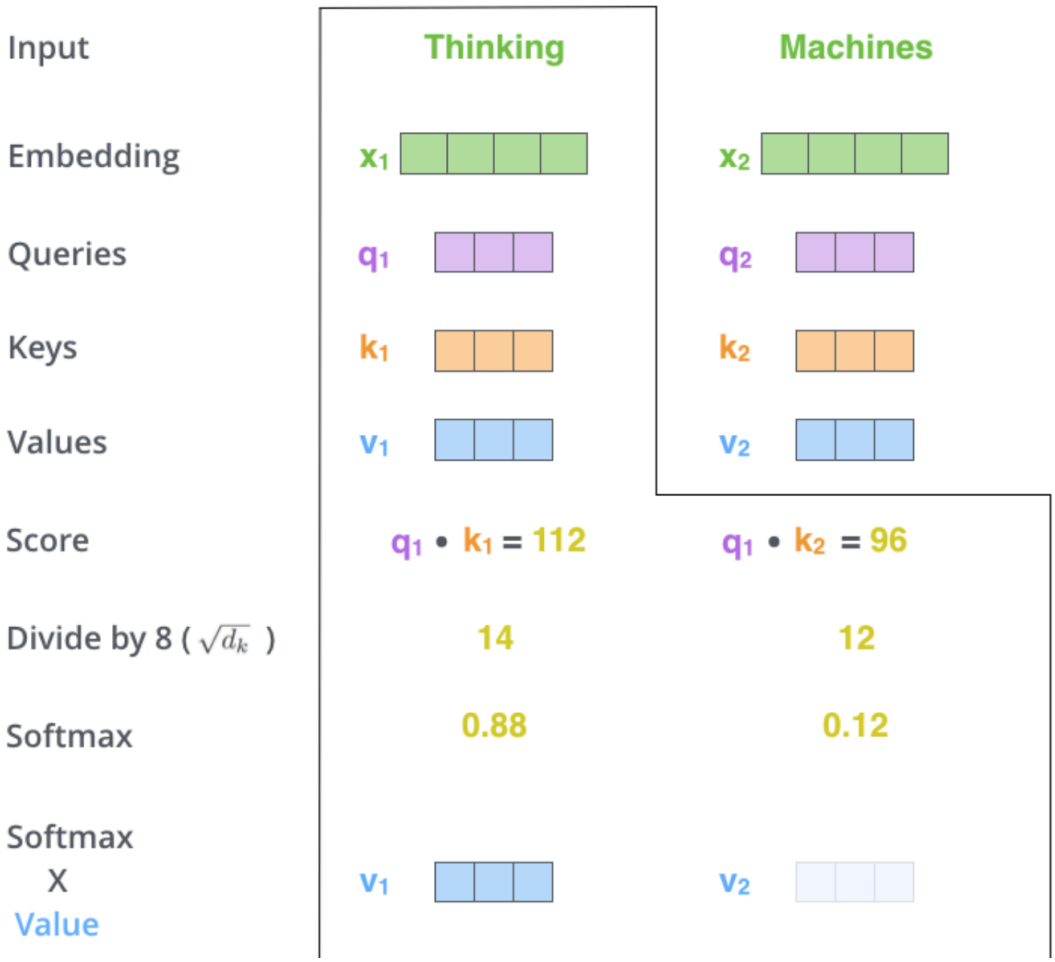



Używamy softmaxa - normalizujemy wartości, żeby wartości były dodatnie i sumowały się do 1.

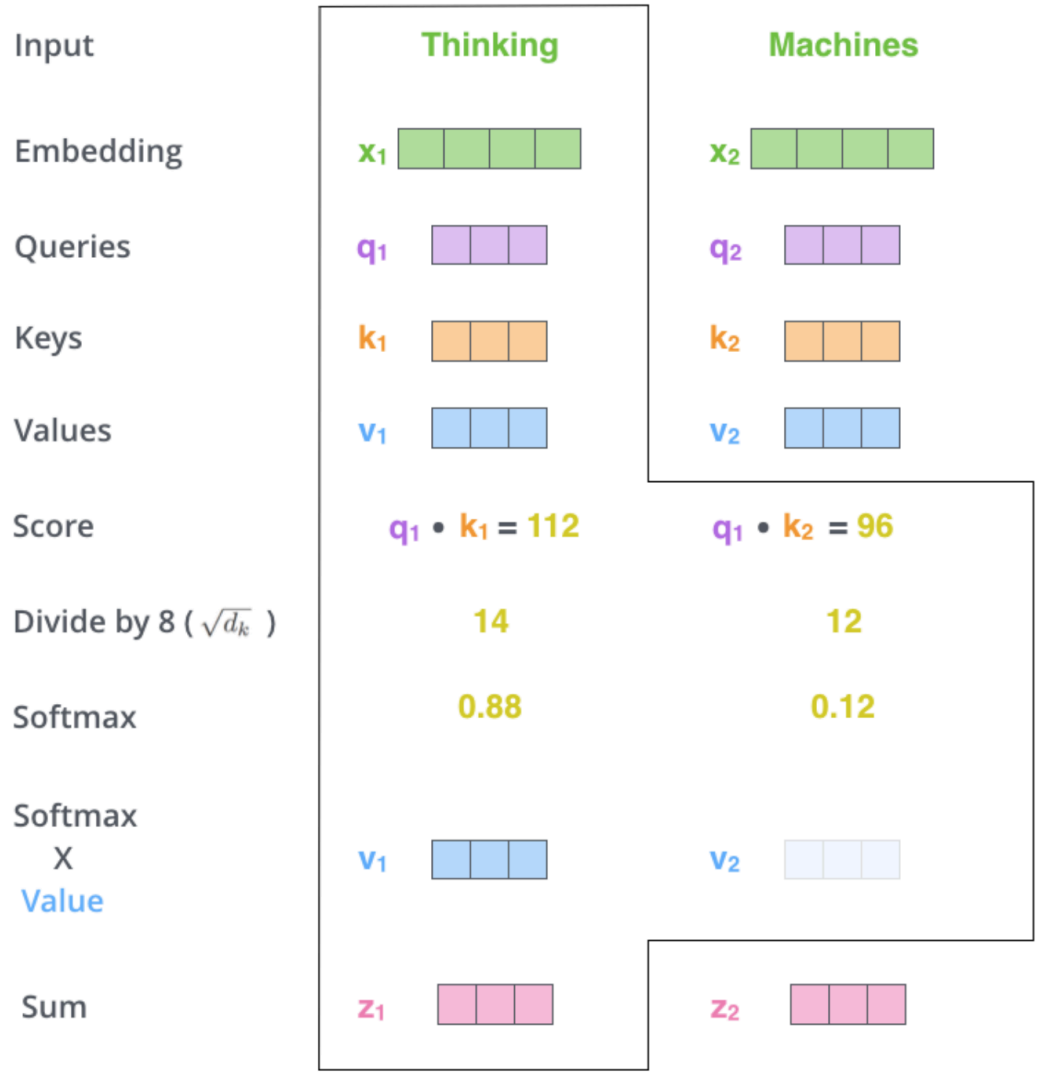



Zauważ, że największy wyniki osiągamy tutaj w liczeniu danego słowa do samego siebie. Jest to naturalne, bo dane słowo reprezentuje siebie najbardziej. Możemy mieć jednak inne słowa, które będą miały wysoki wynik, co oznacza, że mają do siebie jakiś związek (np bird i później w zdaniu it)

Teraz wartość naszego softmaxa przemnażamy przez wektor value. Nie chcemy tutaj naruszyć zbytnio naruszyć wartości naszych słów, któe są dla nas ważne, ale jednocześnie chcemy wyciszyć te słowa, które nic nam nie wnoszą. (Słowa z softmaxem ~0.9 zostaną z dużymi liczbami, natomiast słowa z softmaxem ~0.1 będą miały nikły wpływ)



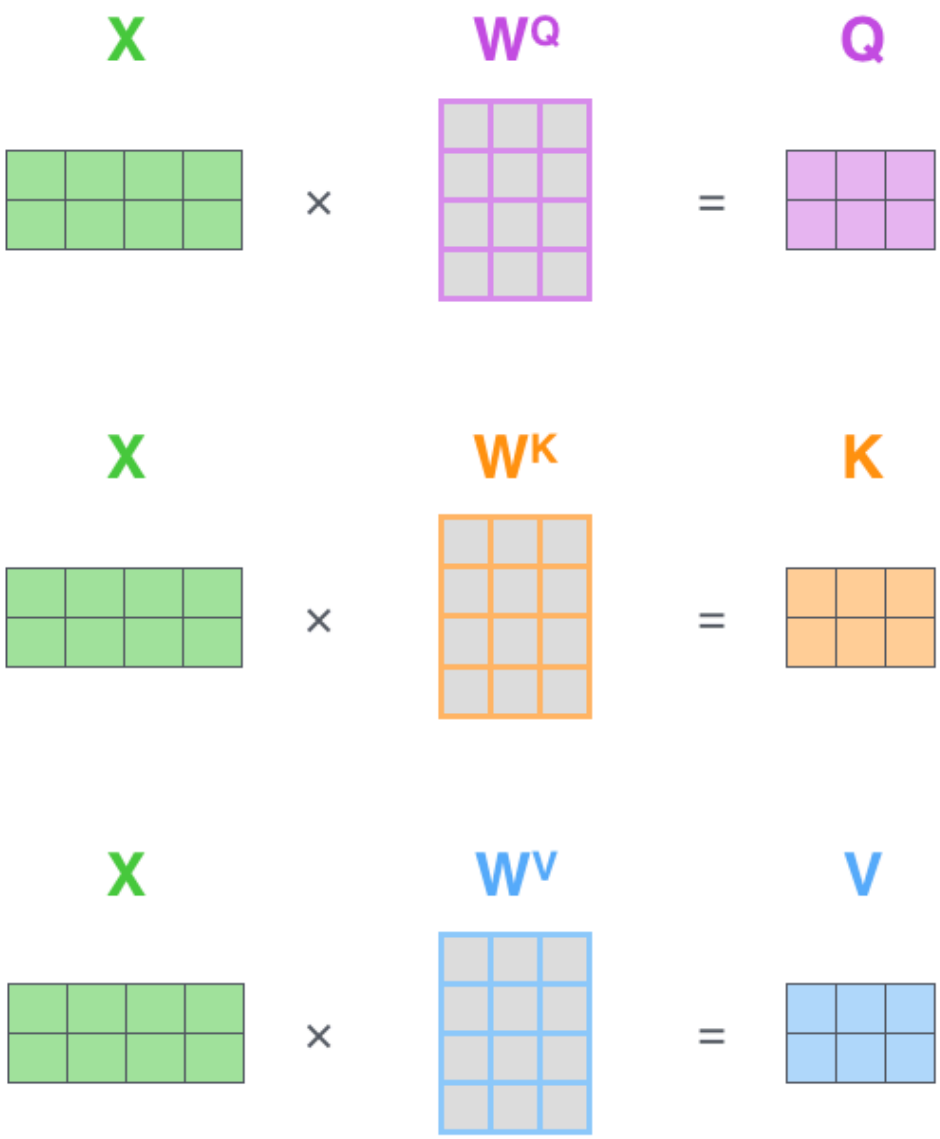

W ten sposób po zebraniu naszego attention w całość, zyskaliśmy nasz nowy wektor, który dalej wyślemy w naszej sieci - do warstwty feed forward. (O dalszej architekturze na kolejnych zajęciach)




## 5 Obliczenie attention - na macierzach

Wektory dobrze wizualizują co siedzieje w attention, ale w rzeczywistym scenariuszu wszystko robimy na macierzach - aby było szybciej.

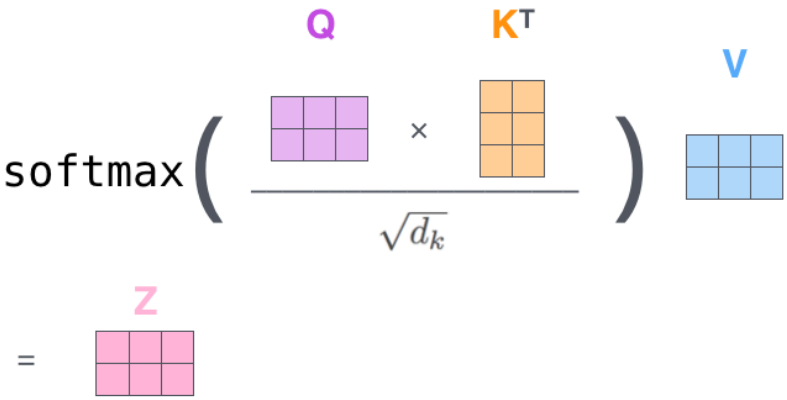
Analogicznie jak z wektorami towrzymy macierze query, key i value:




(każdy wiersz w X odpowiada danemu słowu - to co wcześniej robiliśmy koło siebie, teraz zrobimy "na raz")

W skrócie:

1. Przemnażamy query i key (key jest transponowane, aby macierze się zgadzały)
2. Dzielimy na pierwiastek z wymiaru key (k)
3. Używamy softmaxa
4. Mnożymy razy wartości values
5. Tadam 🐱




In [6]:
```
!pip install numpy matplotlib
```

```
Requirement already satisfied: numpy in /home/sekanko/informatyka/studia/P
JN-cwiczenia/venv/lib64/python3.14/site-packages (2.5.4)
Collecting matplotlib
  Downloading matplotlib-3.11.2-cp314-cp314-manylinux_2_27_x86_64.manylinu
x_2_28_x86_64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.4.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_
2_28_x86_64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.68.0-cp314-cp314-manylinux2014_x86_64.manylinux_
2_17_x86_64.whl.metadata (139 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_
2_17_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: packaging>=20.0 in /home/sekanko/informatyk
a/studia/PJN-cwiczenia/venv/lib64/python3.14/site-packages (from matplotli
b) (26.3)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-13.0.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_
28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: python-dateutil>=2.7 in /home/sekanko/infor
matyka/studia/PJN-cwiczenia/venv/lib64/python3.14/site-packages (from matp
lotlib) (3.0.0)
Requirement already satisfied: six>=1.6 in /home/sekanko/informatyka/studi
a/PJN-cwiczenia/venv/lib64/python3.14/site-packages (from python-dateutil>
=2.7->matplotlib) (1.17.0)
Requirement already satisfied: tzdata in /home/sekanko/informatyka/studia/
PJN-cwiczenia/venv/lib64/python3.14/site-packages (from python-dateutil>=
2.7->matplotlib) (2026.6)
Downloading matplotlib-3.11.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_
2_28_x86_64.whl (11.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.0/11.0 MB 40.3 MB/s  0:00:0
0.8 MB/s eta 0:00:01
Downloading contourpy-1.4.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_
28_x86_64.whl (384 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.68.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_
17_x86_64.whl (5.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.4/5.4 MB 61.9 MB/s  0:00:00
Downloading kiwisolver-1.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_
17_x86_64.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 63.3 MB/s  0:00:00
Downloading pillow-13.0.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28
_x86_64.whl (7.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.1/7.1 MB 56.3 MB/s  0:00:00
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Installing collected packages: pyparsing, pillow, kiwisolver, fonttools, c
ycler, contourpy, matplotlib
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7/7 [matplotlib] 6/7 [matplotl
ib]ow]
Successfully installed contourpy-1.4.0 cycler-0.12.1 fonttools-4.68.0 kiwi
solver-1.5.1 matplotlib-3.11.2 pillow-13.0.0 pyparsing-3.3.3

[notice] A new release of pip is available: 26.0.1 -> 26.2.2
[notice] To update, run: pip install --upgrade pip
```

In [4]:
```python
import numpy as np
import math

def softmax(x, axis=-1):
    """
    Standardowy softmax (stabilny numerycznie).
    Dla każdej linii (np. dla każdego tokena w sekwencji)
    przelicza wartości na rozkład prawdopodobieństwa.
    """
    x_max = np.max(x, axis=axis, keepdims=True)
    e_x = np.exp(x - x_max)  # stabilizacja numeryczna
    return e_x / np.sum(e_x, axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None, verbose=True):
    """
    Implementacja mechanizmu Scaled Dot-Product Attention krok po kroku.

    Parametry:
    -----------
    Q : np.ndarray, shape (..., seq_len_q, dk)
        Zapytania (queries)
    K : np.ndarray, shape (..., seq_len_k, dk)
        Klucze (keys)
    V : np.ndarray, shape (..., seq_len_k, dv)
        Wartości (values)
    mask : np.ndarray lub None
        Maska (np. do maskowania paddingu lub przyszłych tokenów)
    verbose : bool
        Czy drukować pośrednie wyniki

    Zwraca:
    --------
    output : (..., seq_len_q, dv)
        Wynik ważonej sumy wartości
    attn_weights : (..., seq_len_q, seq_len_k)
        Wagi uwagi po softmaxie
    """

    dk = Q.shape[-1]

    if verbose:
        print("==== INPUTS ====")
        print(f"Q shape: {Q.shape}")
        print(f"K shape: {K.shape}")
        print(f"V shape: {V.shape}\n")
        print("Q =", np.round(Q, 3))
        print("K =", np.round(K, 3))
        print("V =", np.round(V, 3))
        print("================\n")

    # Liczymy „scores” – czyli miarę podobieństwa zapytań Q i kluczy K
    scores = np.matmul(Q, K.transpose(0, 2, 1)) if Q.ndim == 3 else np.ma
    if verbose:
        print("Scores (QK^T):")
        print(np.round(scores, 3), "\n")

    # Skalowanie przez sqrt(dk)
    scores = scores / math.sqrt(dk)
    if verbose:
```

```
        print(f"Scaled scores (dzielone przez sqrt({dk}) = {math.sqrt(dk)
        print(np.round(scores, 3), "\n")

    # Dodatke: Dodajemy maskę (jeśli jest)
    if mask is not None:
        scores = scores + mask
        if verbose:
            print("After adding mask:")
            print(np.round(scores, 3), "\n")

    # Softmax – zamienia scores na rozkład wag uwagi
    attn_weights = softmax(scores, axis=-1)
    if verbose:
        print("Attention weights po softmax:")
        print(np.round(attn_weights, 3))
        print(f"Każdy wiersz sumuje się do: {np.sum(attn_weights, axis=-1

    # Mnożymy przez V, żeby uzyskać ważoną sumę wartości
    output = np.matmul(attn_weights, V)
    if verbose:
        print("Wynik po pomnożeniu przez V:")
        print(np.round(output, 3))
        print("============================\n")

    return output, attn_weights


# 🔍 Quick sanity check
np.random.seed(0)
Q = np.random.rand(1, 3, 4)  # batch=1, query len=3, dk=4
K = np.random.rand(1, 5, 4)  # key len=5
V = np.random.rand(1, 5, 6)  # value dim=6

out, w = scaled_dot_product_attention(Q, K, V)

print("FINAL OUTPUT SHAPES:")
print("Output shape:", out.shape)
print("Weights shape:", w.shape)
```

```
==== INPUTS ====
Q shape: (1, 3, 4)
K shape: (1, 5, 4)
V shape: (1, 5, 6)

Q = [[[0.549 0.715 0.603 0.545]
  [0.424 0.646 0.438 0.892]
  [0.964 0.383 0.792 0.529]]]
K = [[[0.568 0.926 0.071 0.087]
  [0.02  0.833 0.778 0.87 ]
  [0.979 0.799 0.461 0.781]
  [0.118 0.64  0.143 0.945]
  [0.522 0.415 0.265 0.774]]]
V = [[[0.456 0.568 0.019 0.618 0.612 0.617]
  [0.944 0.682 0.36  0.437 0.698 0.06 ]
  [0.667 0.671 0.21  0.129 0.315 0.364]
  [0.57  0.439 0.988 0.102 0.209 0.161]
  [0.653 0.253 0.466 0.244 0.159 0.11 ]]]
================

Scores (QK^T):
[[[1.064 1.55  1.812 1.124 1.164]
  [0.947 1.663 1.829 1.369 1.295]
  [1.005 1.415 2.028 0.972 1.281]]]

Scaled scores (dzielone przez sqrt(4) = 2.000):
[[[0.532 0.775 0.906 0.562 0.582]
  [0.474 0.831 0.914 0.684 0.648]
  [0.502 0.707 1.014 0.486 0.64 ]]]

Attention weights po softmax:
[[[0.172 0.219 0.25  0.177 0.181]
  [0.156 0.223 0.242 0.193 0.186]
  [0.166 0.204 0.277 0.163 0.19 ]]]
Każdy wiersz sumuje się do: [[1. 1. 1.]]

Wynik po pomnożeniu przez V:
[[[0.672 0.539 0.394 0.297 0.403 0.259]
  [0.675 0.535 0.411 0.29  0.397 0.249]
  [0.67  0.539 0.385 0.29  0.395 0.263]]]
============================

FINAL OUTPUT SHAPES:
Output shape: (1, 3, 6)
Weights shape: (1, 3, 5)
```

In [7]:

```python
# SELF-ATTENTION DLA JEDNEGO ZDANIA

import numpy as np
import matplotlib.pyplot as plt
import math

# softmax i funkcja attention
def softmax(x, axis=-1):
    x_max = np.max(x, axis=axis, keepdims=True)
    e_x = np.exp(x - x_max)
    return e_x / np.sum(e_x, axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    dk = Q.shape[-1]
```

```
    scores = np.matmul(Q, K.transpose(0, 2, 1)) / math.sqrt(dk)
    if mask is not None:
        scores = scores + mask
    attn_weights = softmax(scores, axis=-1)
    output = np.matmul(attn_weights, V)
    return output, attn_weights


# Dane wejściowe (symulacja)
np.random.seed(42)

tokens = ["I", "like", "big", "cats"]
seq_len = len(tokens)
d_k = 4
d_v = 6

Q = np.random.rand(1, seq_len, d_k)
K = np.random.rand(1, seq_len, d_k)
V = np.random.rand(1, seq_len, d_v)

print("Q shape:", Q.shape)
print("K shape:", K.shape)
print("V shape:", V.shape)

# Obliczenie uwagi
out, attn_weights = scaled_dot_product_attention(Q, K, V)

print("\nAttention weights shape:", attn_weights.shape)
print("Każdy wiersz sumuje się do 1 →", np.sum(attn_weights, axis=-1))

# Wizualizacja wag uwagi
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(attn_weights[0], cmap="Blues")
ax.set_xticks(np.arange(seq_len))
ax.set_yticks(np.arange(seq_len))
ax.set_xticklabels(tokens)
ax.set_yticklabels(tokens)
plt.setp(ax.get_xticklabels(), rotation=45, ha="right", rotation_mode="an
ax.set_xlabel("Na które słowa patrzymy (Keys)")
ax.set_ylabel("Które słowo patrzy (Queries)")
ax.set_title("🧠 Self-Attention: 'I like big cats'")

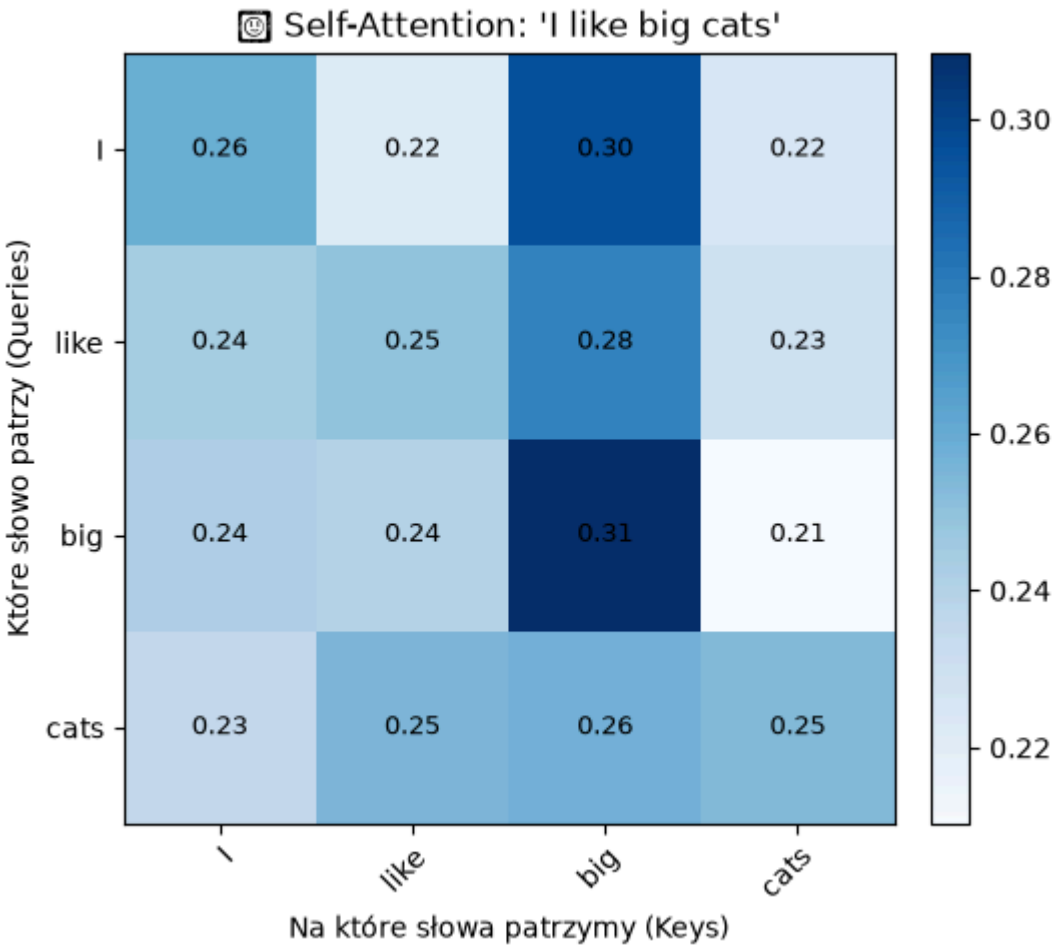
for i in range(seq_len):
    for j in range(seq_len):
        text = ax.text(j, i, f"{attn_weights[0, i, j]:.2f}",
                       ha="center", va="center", color="black", fontsize=

plt.colorbar(im, fraction=0.046, pad=0.04)
plt.show()
```

```
Q shape: (1, 4, 4)
K shape: (1, 4, 4)
V shape: (1, 4, 6)

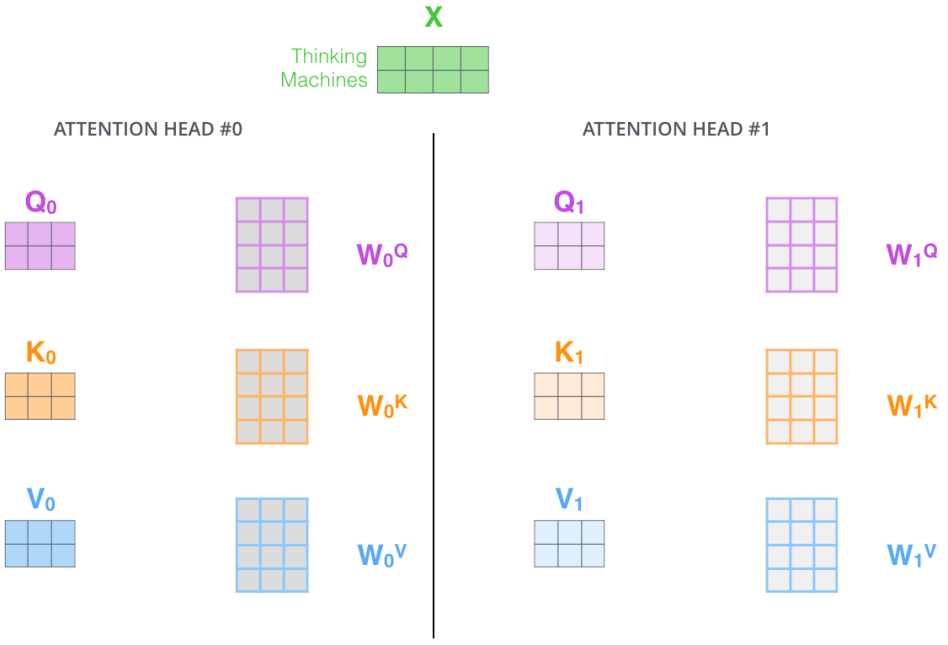
Attention weights shape: (1, 4, 4)
Każdy wiersz sumuje się do 1 → [[1. 1. 1. 1.]]
```

```
/home/sekanko/informatyka/studia/PJN-cwiczenia/venv/lib64/python3.14/site-
packages/IPython/core/pylabtools.py:158: UserWarning: Glyph 129504 (\N{BRA
IN}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
```




# Multi-Head Attention

Jakby sam mechanizm uwagi był za prosty :) W transformerach mozemy spotkać coś co nazywa się Multi-Head Attention




Jest to rozwinięcie samego attention, która pozwala modelowi uczyć się wielu sposobów "patrzenia" na kontekst słów w zdaniu.

Zamiast jednej atencji (jednego zestawu Q, K, V), tworzymy ich kilka (np. 8), a następnie łączymy wyniki w jedną reprezentację.

In [8]:
```
!pip install torch
```

```
Collecting torch
  Downloading torch-2.15.0-cp314-cp314-manylinux_2_28_x86_64.whl.metadata
(36 kB)
Collecting filelock (from torch)
  Downloading filelock-4.2.4-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: typing-extensions>=4.13.0 in /home/sekanko/
informatyka/studia/PJN-cwiczenia/venv/lib64/python3.14/site-packages (from
torch) (4.16.0)
Collecting setuptools>=77.0.3 (from torch)
  Downloading setuptools-84.2.1-py3-none-any.whl.metadata (6.6 kB)
Collecting sympy>=1.13.3 (from torch)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch)
  Downloading networkx-3.7-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: jinja2 in /home/sekanko/informatyka/studia/
PJN-cwiczenia/venv/lib64/python3.14/site-packages (from torch) (3.1.6)
Collecting fsspec>=0.8.5 (from torch)
  Downloading fsspec-2026.9.0-py3-none-any.whl.metadata (10 kB)
Collecting cuda-toolkit==13.2.2 (from cuda-toolkit[cublas,cudart,cufft,cuf
ile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platform_system
== "Linux"->torch)
  Downloading cuda_toolkit-13.2.2-py2.py3-none-any.whl.metadata (20 kB)
Collecting cuda-bindings<14,>=13.0.3 (from torch)
  Downloading cuda_bindings-13.4.4-cp314-cp314-manylinux_2_24_x86_64.manyl
inux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting nvidia-cudnn-cu13==9.26.0.51 (from torch)
  Downloading nvidia_cudnn_cu13-9.26.0.51-py3-none-manylinux_2_27_x86_64.w
hl.metadata (1.9 kB)
Collecting nvidia-cusparselt-cu13==0.8.1 (from torch)
  Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-manylinux2014_x86_64.w
hl.metadata (12 kB)
Collecting nvidia-nccl-cu13==2.30.7 (from torch)
  Downloading nvidia_nccl_cu13-2.30.7-py3-none-manylinux_2_18_x86_64.whl.m
etadata (2.1 kB)
Collecting nvidia-nvshmem-cu13==3.7.2 (from torch)
  Downloading nvidia_nvshmem_cu13-3.7.2-py3-none-manylinux2014_x86_64.many
linux_2_17_x86_64.whl.metadata (2.2 kB)
Collecting triton~=3.9.0 (from torch)
  Downloading triton-3.9.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28
_x86_64.whl.metadata (978 bytes)
Collecting nvidia-cublas==13.4.1.3.* (from cuda-toolkit[cublas,cudart,cuff
t,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platform_s
ystem == "Linux"->torch)
  Downloading nvidia_cublas-13.4.1.3-py3-none-manylinux_2_27_x86_64.whl.me
tadata (1.8 kB)
Collecting nvidia-cuda-nvrtc==13.2.86.* (from cuda-toolkit[cublas,cudart,c
ufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platfor
m_system == "Linux"->torch)
  Downloading nvidia_cuda_nvrtc-13.2.86-py3-none-manylinux2010_x86_64.many
linux_2_12_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cuda-runtime==13.2.86.* (from cuda-toolkit[cublas,cudar
t,cufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; plat
form_system == "Linux"->torch)
  Downloading nvidia_cuda_runtime-13.2.86-py3-none-manylinux2014_x86_64.ma
nylinux_2_17_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cufft==12.2.0.57.* (from cuda-toolkit[cublas,cudart,cuff
t,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platform_s
ystem == "Linux"->torch)
  Downloading nvidia_cufft-12.2.0.57-py3-none-manylinux2014_x86_64.manylin
ux_2_17_x86_64.whl.metadata (1.8 kB)
```

```
Collecting nvidia-nvjitlink<14,>=13.2.86 (from cuda-toolkit[cublas,cudart,
cufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platfo
rm_system == "Linux"->torch)
  Downloading nvidia_nvjitlink-13.4.92-py3-none-manylinux2010_x86_64.manyl
inux_2_12_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cufile==1.17.1.22.* (from cuda-toolkit[cublas,cudart,cuf
ft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platform_
system == "Linux"->torch)
  Downloading nvidia_cufile-1.17.1.22-py3-none-manylinux_2_27_x86_64.whl.m
etadata (1.8 kB)
Collecting nvidia-cuda-cupti==13.2.86.* (from cuda-toolkit[cublas,cudart,c
ufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platfor
m_system == "Linux"->torch)
  Downloading nvidia_cuda_cupti-13.2.86-py3-none-manylinux_2_25_x86_64.wh
l.metadata (1.8 kB)
Collecting nvidia-cusolver==12.2.0.11.* (from cuda-toolkit[cublas,cudart,c
ufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platfor
m_system == "Linux"->torch)
  Downloading nvidia_cusolver-12.2.0.11-py3-none-manylinux_2_27_x86_64.wh
l.metadata (1.9 kB)
Collecting nvidia-cusparse==12.7.10.12.* (from cuda-toolkit[cublas,cudart,
cufft,cufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platfo
rm_system == "Linux"->torch)
  Downloading nvidia_cusparse-12.7.10.12-py3-none-manylinux2014_x86_64.man
ylinux_2_17_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvtx==13.2.86.* (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.2.2; platform_syst
em == "Linux"->torch)
  Downloading nvidia_nvtx-13.2.86-py3-none-manylinux1_x86_64.manylinux_2_5
_x86_64.whl.metadata (1.9 kB)
Collecting nvidia-cuda-cccl (from nvidia-nvshmem-cu13==3.7.2->torch)
  Downloading nvidia_cuda_cccl-13.3.4.3.1-py3-none-manylinux2014_x86_64.ma
nylinux_2_17_x86_64.whl.metadata (1.7 kB)
Collecting cuda-pathfinder>=1.4.2 (from cuda-bindings<14,>=13.0.3->torch)
  Downloading cuda_pathfinder-1.8.4-py3-none-any.whl.metadata (1.9 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /home/sekanko/informatyk
a/studia/PJN-cwiczenia/venv/lib64/python3.14/site-packages (from jinja2->t
orch) (3.0.4)
Downloading torch-2.15.0-cp314-cp314-manylinux_2_28_x86_64.whl (539.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 539.2/539.2 MB 40.4 MB/s  0:0
0:14 eta 0:00:01[36m0:00:01
Downloading cuda_toolkit-13.2.2-py2.py3-none-any.whl (2.6 kB)
Downloading nvidia_cudnn_cu13-9.26.0.51-py3-none-manylinux_2_27_x86_64.whl
(519.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 519.2/519.2 MB 30.1 MB/s  0:0
0:13 eta 0:00:01[36m0:00:01
Downloading nvidia_cusparselt_cu13-0.8.1-py3-none-manylinux2014_x86_64.whl
(170.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 170.1/170.1 MB 30.1 MB/s  0:0
0:05 eta 0:00:01[36m0:00:01
Downloading nvidia_nccl_cu13-2.30.7-py3-none-manylinux_2_18_x86_64.whl (21
6.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 216.0/216.0 MB 36.2 MB/s  0:0
0:05 eta 0:00:01[36m0:00:01
Downloading nvidia_nvshmem_cu13-3.7.2-py3-none-manylinux2014_x86_64.manyli
nux_2_17_x86_64.whl (135.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 135.4/135.4 MB 38.8 MB/s  0:0
0:03 eta 0:00:010:01:01
```

```
Downloading cuda_bindings-13.4.4-cp314-cp314-manylinux_2_24_x86_64.manylin
ux_2_28_x86_64.whl (7.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 44.3 MB/s  0:00:00
Downloading nvidia_cublas-13.4.1.3-py3-none-manylinux_2_27_x86_64.whl (40
4.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 404.9/404.9 MB 40.8 MB/s  0:0
0:09 eta 0:00:010:01:02
Downloading nvidia_cuda_cupti-13.2.86-py3-none-manylinux_2_25_x86_64.whl
(12.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 18.2 MB/s  0:00:0
0a 0:00:01[36m0:00:01
Downloading nvidia_cuda_nvrtc-13.2.86-py3-none-manylinux2010_x86_64.manyli
nux_2_12_x86_64.whl (47.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.0/47.0 MB 31.5 MB/s  0:00:0
1 eta 0:00:010:01
Downloading nvidia_cuda_runtime-13.2.86-py3-none-manylinux2014_x86_64.many
linux_2_17_x86_64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 18.2 MB/s  0:00:00
Downloading nvidia_cufft-12.2.0.57-py3-none-manylinux2014_x86_64.manylinux
_2_17_x86_64.whl (218.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 218.3/218.3 MB 49.9 MB/s  0:0
0:04 eta 0:00:01[36m0:00:01
Downloading nvidia_cufile-1.17.1.22-py3-none-manylinux_2_27_x86_64.whl (1.
4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 15.6 MB/s  0:00:00
Downloading nvidia_cusolver-12.2.0.11-py3-none-manylinux_2_27_x86_64.whl
(227.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 227.9/227.9 MB 37.6 MB/s  0:0
0:06 eta 0:00:01[36m0:00:01
Downloading nvidia_cusparse-12.7.10.12-py3-none-manylinux2014_x86_64.manyl
inux_2_17_x86_64.whl (151.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 151.0/151.0 MB 32.1 MB/s  0:0
0:04 eta 0:00:01[36m0:00:01
Downloading nvidia_nvjitlink-13.4.92-py3-none-manylinux2010_x86_64.manylin
ux_2_12_x86_64.whl (42.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 42.5/42.5 MB 35.7 MB/s  0:00:0
17.7 MB/s eta 0:00:01:01
Downloading nvidia_nvtx-13.2.86-py3-none-manylinux1_x86_64.manylinux_2_5_x
86_64.whl (153 kB)
Downloading triton-3.9.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x
86_64.whl (285.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 285.7/285.7 MB 36.1 MB/s  0:0
0:07 eta 0:00:010:01:01
Downloading cuda_pathfinder-1.8.4-py3-none-any.whl (62 kB)
Downloading fsspec-2026.9.0-py3-none-any.whl (221 kB)
Downloading networkx-3.7-py3-none-any.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 18.7 MB/s  0:00:00
Downloading setuptools-84.2.1-py3-none-any.whl (876 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 876.3/876.3 kB 9.5 MB/s  0:00:
00
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.3/6.3 MB 14.5 MB/s  0:00:00
eta 0:00:01
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 536.2/536.2 kB 7.0 MB/s  0:00:
00
Downloading filelock-4.2.4-py3-none-any.whl (162 kB)
Downloading nvidia_cuda_cccl-13.3.4.3.1-py3-none-manylinux2014_x86_64.many
linux_2_17_x86_64.whl (3.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.9/3.9 MB 14.1 MB/s  0:00:00
```

```
3.6 MB/s eta 0:00:01
Installing collected packages: nvidia-cusparselt-cu13, mpmath, cuda-toolki
t, triton, sympy, setuptools, nvidia-nvtx, nvidia-nvjitlink, nvidia-nccl-c
u13, nvidia-cufile, nvidia-cuda-runtime, nvidia-cuda-nvrtc, nvidia-cuda-cu
pti, nvidia-cuda-cccl, networkx, fsspec, filelock, cuda-pathfinder, nvidia
-nvshmem-cu13, nvidia-cusparse, nvidia-cufft, nvidia-cublas, cuda-binding
s, nvidia-cusolver, nvidia-cudnn-cu13, torch
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26/26 [torch]7m━ 25/26 [torch]
[nvidia-cudnn-cu13]3]
Successfully installed cuda-bindings-13.4.4 cuda-pathfinder-1.8.4 cuda-too
lkit-13.2.2 filelock-4.2.4 fsspec-2026.9.0 mpmath-1.3.0 networkx-3.7 nvidi
a-cublas-13.4.1.3 nvidia-cuda-cccl-13.3.4.3.1 nvidia-cuda-cupti-13.2.86 nv
idia-cuda-nvrtc-13.2.86 nvidia-cuda-runtime-13.2.86 nvidia-cudnn-cu13-9.2
6.0.51 nvidia-cufft-12.2.0.57 nvidia-cufile-1.17.1.22 nvidia-cusolver-12.
2.0.11 nvidia-cusparse-12.7.10.12 nvidia-cusparselt-cu13-0.8.1 nvidia-nccl
-cu13-2.30.7 nvidia-nvjitlink-13.4.92 nvidia-nvshmem-cu13-3.7.2 nvidia-nvt
x-13.2.86 setuptools-84.2.1 sympy-1.14.0 torch-2.15.0 triton-3.9.0

[notice] A new release of pip is available: 26.0.1 -> 26.2.2
[notice] To update, run: pip install --upgrade pip
```

In [9]:
```python
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
```

Out[9]:
```
<torch._C.Generator at 0x7efdf4ee3cd0>
```

## 1 Wejście

Wyobraźmy sobie zdanie: "The animal didn't cross the street because it was too tired".

Dla uproszczenia stwórzmy losowe wektory (embeddingi) dla 12 słów.

In [10]:
```python
seq_len = 12   # liczba słów
d_model = 64  # rozmiar wektora (embedding)
x = torch.rand(seq_len, d_model)
print("Kształt wejścia:", x.shape)
```

```
Kształt wejścia: torch.Size([12, 64])
```

# 2 Attention (Single-Head)

Przekształca każde słowo (wektor) w nowe reprezentacje używając trzech macierzy wag: W_Q, W_K, W_V (to co wcześniej)

In [11]:
```python
d_k = d_model  # dla prostoty
W_Q = nn.Linear(d_model, d_k)
W_K = nn.Linear(d_model, d_k)
W_V = nn.Linear(d_model, d_k)

Q = W_Q(x)
K = W_K(x)
V = W_V(x)
```

```
print("Kształty Q, K, V:", Q.shape, K.shape, V.shape)
```

```
Kształty Q, K, V: torch.Size([12, 64]) torch.Size([12, 64]) torch.Size([1
2, 64])
```

In [12]:
```
scores = torch.matmul(Q, K.T) / torch.sqrt(torch.tensor(d_k, dtype=torch.
attn_weights = F.softmax(scores, dim=-1)
Z = torch.matmul(attn_weights, V)

print("Kształt macierzy wynikowej Z (jedna głowa):", Z.shape)
```

```
Kształt macierzy wynikowej Z (jedna głowa): torch.Size([12, 64])
```

## 3 Multi head attention

Teraz zamiast jednej atencji, tworzymy kilka "głów" — każda z własnymi W_Q, W_K, W_V. Każda głowa "patrzy" na dane w inny sposób

In [13]:
```
num_heads = 8
head_dim = d_model // num_heads  # 64 rozmiar modelu / 8 głow = 8 rozmiar

W_Q = nn.Linear(d_model, d_model)
W_K = nn.Linear(d_model, d_model)
W_V = nn.Linear(d_model, d_model)

Q = W_Q(x).view(seq_len, num_heads, head_dim)
K = W_K(x).view(seq_len, num_heads, head_dim)
V = W_V(x).view(seq_len, num_heads, head_dim)

print("Kształt Q po podziale na głowy:", Q.shape)  # [12, 8, 8]
```

```
Kształt Q po podziale na głowy: torch.Size([12, 8, 8])
```

Obliczamy uwagę osobno dla każdej głowy

In [14]:
```
Z_heads = []
for h in range(num_heads):
    q, k, v = Q[:, h, :], K[:, h, :], V[:, h, :]
    scores = torch.matmul(q, k.T) / torch.sqrt(torch.tensor(head_dim, dty
    attn = F.softmax(scores, dim=-1)
    z = torch.matmul(attn, v)
    Z_heads.append(z)

print("Z_heads:", [h.shape for h in Z_heads], "Len Z_heads:", len(Z_heads

# [12, 64] po połączeniu wszystkich głów - to co mieliśmy przed
Z_multi = torch.cat(Z_heads, dim=-1)
print("Z po połączeniu wszystkich głów:", Z_multi.shape)
```

```
Z_heads: [torch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8]), t
orch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8]), torch.Size
([12, 8]), torch.Size([12, 8])] Len Z_heads: 8
Z po połączeniu wszystkich głów: torch.Size([12, 64])
```

Każda głowa:

- z_i.shape = [12, 8] → czyli 12 tokenów × 8 cech (bo head_dim = 8).

Mamy 8 głów * 8 cech = 64 → To jest surowe połączenie wyników głów — wszystkie perspektywy „obok siebie".

Warstwa łączenia (Output Projection) - Po złączeniu wszystkich głów, mnożymy przez macierz WO, żeby wrócić do wymiaru d_model. To przygotowuje dane do kolejnych warstw transformera.

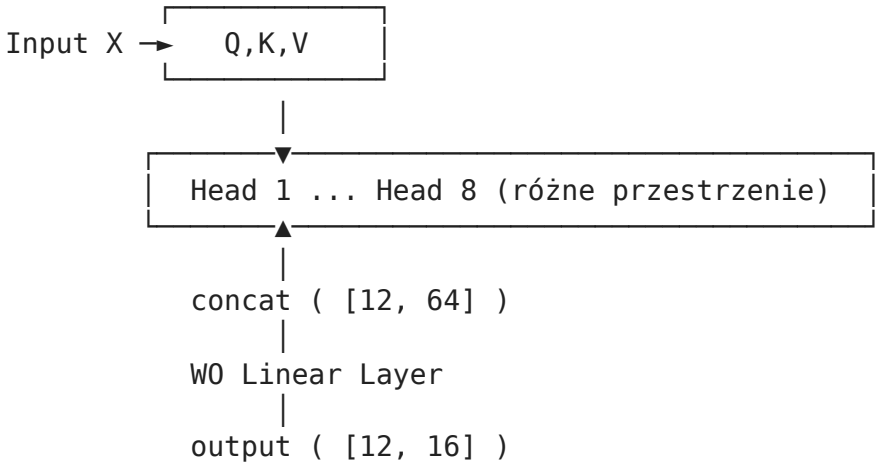
In [15]:
```
W_O = nn.Linear(d_model, d_model)
output = W_O(Z_multi)
print("Ostateczny kształt po WO:", output.shape)
```

```
Ostateczny kształt po WO: torch.Size([12, 64])
```

Dlaczego nie kończymy na łączeniu wszystkich głów?

Bo bez W_O:

- każda głowa miałaby „swoje" cechy, ale nie byłoby interakcji między nimi, sieć dalej miałaby tensor [12, 64] — niezgodny z oczekiwanym d_model.
- W_O gwarantuje, że każdy token po multi-head attention ma z powrotem wektor o tym samym rozmiarze, co na wejściu (d_model) — dzięki czemu architektura Transformera pozostaje spójna w każdej warstwie.




Chwila, chwila.. przecież mieliśmy wymiar [12, 64] po złączeniu głów i po warstwie łaczenia, która ma wrócić nas do wymiaru d_model mamy dokładnie ten sam wymiar?! Co to za oszustwa 😠

Jeśli w modelu:

In [16]:
```
d_model == num_heads * head_dim
```

Out[16]:
```
True
```

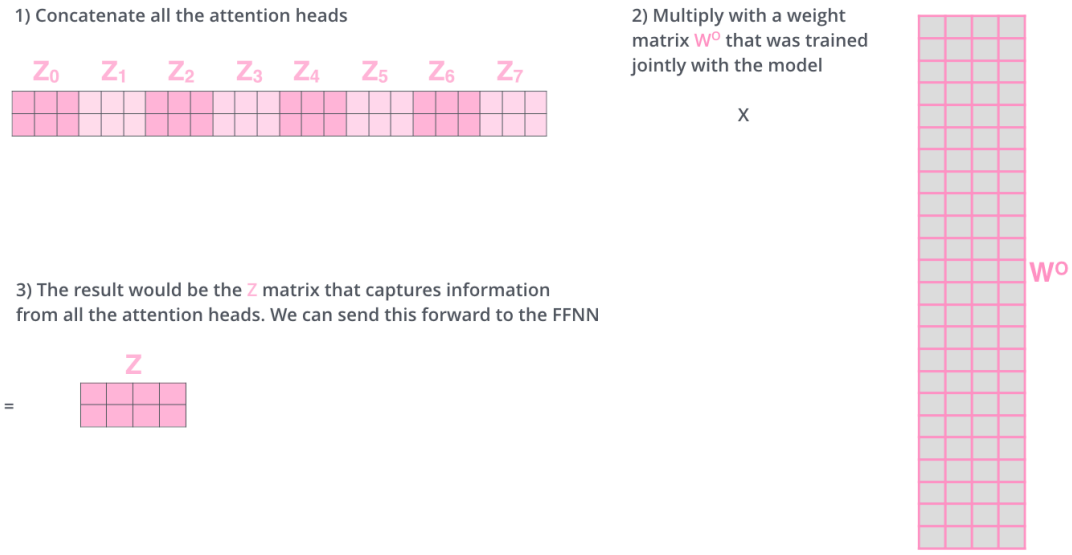
to:

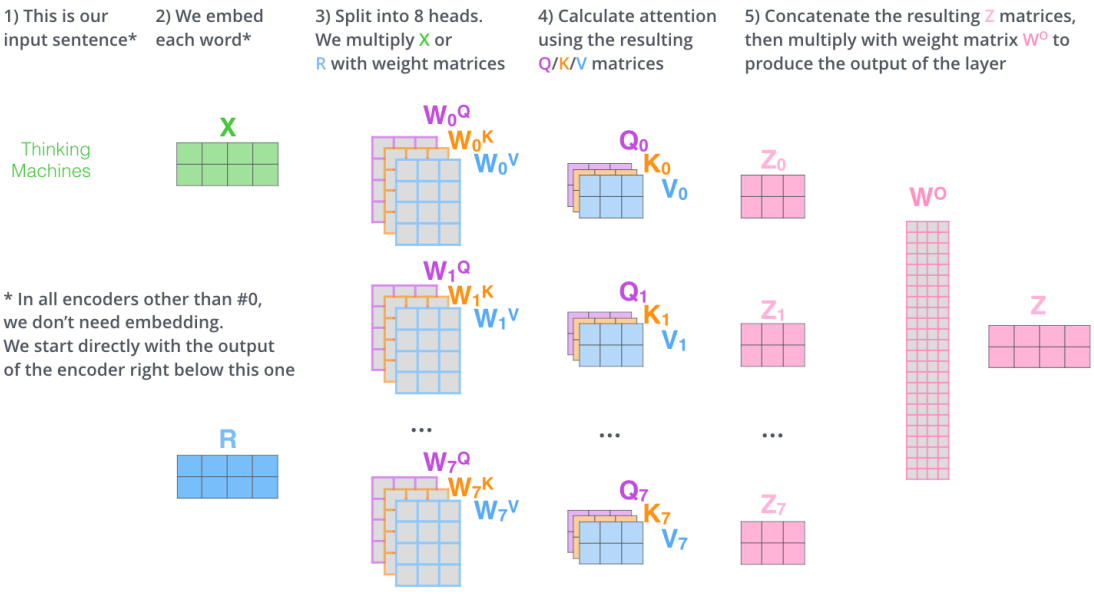
- po konkatenacji (concat) masz [seq_len, 64]

- po W_O dalej masz [seq_len, 64]

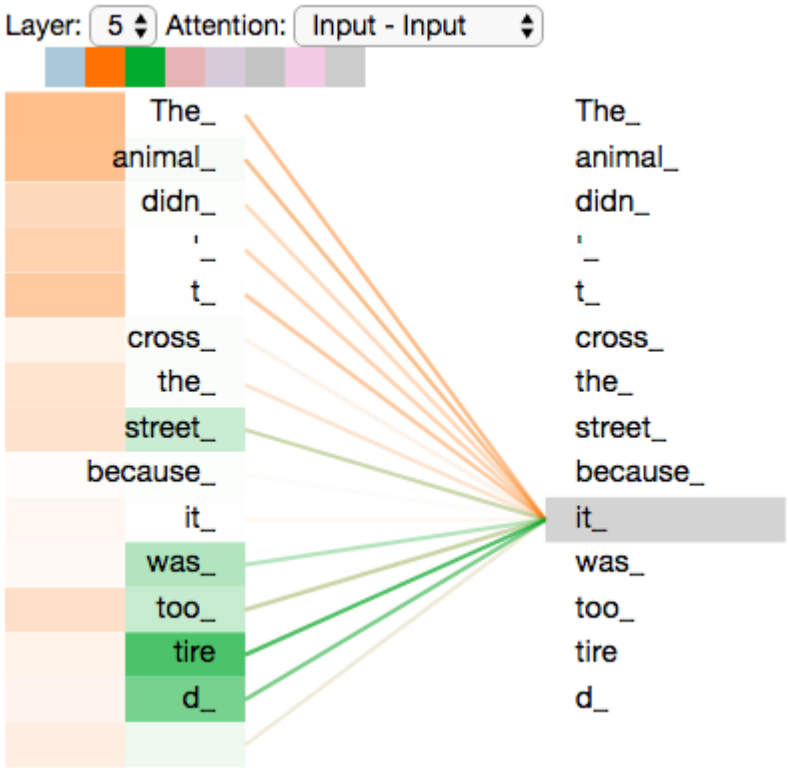
Więc — tak, wymiar liczbowo się nie zmienia.

W_O **nie** istnieje po to, żeby zmienić rozmiar tensora. Istnieje po to, żeby nauczyć się **mieszać informacje z różnych głów.**

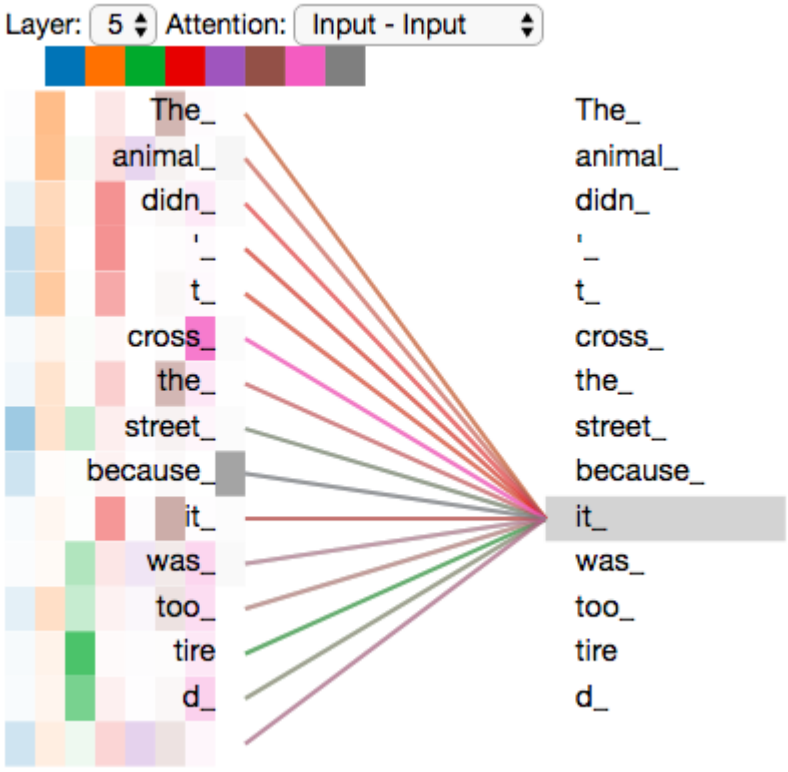



Teraz możemy porównać jak przykładowe dwie głowy traktują nasze zdanie wyjściowe:




Ale po dodaniu 8 głów:




# Zadania

## Zadanie 1

Liczymy attention:

1. Utwórz wektory:

- Q = [1, 0]
- K1 = [1, 1]
- K2 = [0, 1]
- V1 = [2, 0]
- V2 = [0, 2]

2. Oblicz podobieństwo - score1 i score2: Q * K^T

3. Skalujemy - scaled1 i scaled2: dzielenie przez sqrt(d_k), uznajmy że d_k = 2

4. zastosuj softmax - probs

(Podpowiedź: To już nie będzie pojedynczy numerek a wektor: `scaled_scores = np.array([scaled1, scaled2])` i na tym robimy softmax )

5. Suma ważona Values - output

6. Odpowiedz na pytanie zawarte w print - które słowo (Key) miało największy wpływ na wynik?

In [25]:
```python
# 0) Dane
Q = np.array([1, 0], dtype=float)
K1 = np.array([1, 1], dtype=float)
K2 = np.array([0, 1], dtype=float)
V1 = np.array([2, 0], dtype=float)
V2 = np.array([0, 2], dtype=float)
d_k = 2

# 1) podobieństwa
score1 = np.dot(Q, K1)
score2 = np.dot(Q, K2)

# 2) scaled scores
scaled1 = score1 / math.sqrt(d_k)
scaled2 = score2 / math.sqrt(d_k)
scaled_scores = np.array([scaled1, scaled2])

# 3) softmax
def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

probs = softmax(scaled_scores)

# 4) weighted sum of V
output = probs[0] * V1 + probs[1] * V2

# Print zadanie 1
print("Scores:", np.array([score1, score2]))
print("Scaled scores:", scaled_scores)
print("Softmax probabilities:", probs)
```

```
print("Output attention (wektor):", output)
print("Interpretacja: Największy wpływ na wynik miał klucz K1, któremu mo
```

```
Scores: [1. 0.]
Scaled scores: [0.70710678 0.        ]
Softmax probabilities: [0.66976155 0.33023845]
Output attention (wektor): [1.3395231 0.6604769]
Interpretacja: Największy wpływ na wynik miał klucz K1, któremu model przy
pisał większą wagę uwagi niż kluczowi K2. Zapytanie Q jest bardziej podobn
e do K1, co daje wyższy iloczyn skalarny, a wektor wyjściowy znajduje się
w efekcie bliżej V1. Należy jednak pamiętać, że jest to przykład na bardzo
małych, ręcznie dobranych wektorach.
```

# Zadanie 2

Utwórz prostą implementację multi-head attention dla dwóch głów, korzystając z z Zadania 1.

Każda głowa będzie miała własne wektory Qi, Ki, Vi, maciesze Wq, Wk i Wv oraz własne wyniki softmax i output.

Na końcu połączysz wyniki obu głów (konkatenacja), aby uzyskać końcowy wektor.

Zwizualizuj wagi uwagi - forma dowolna

In [40]:
```
import numpy as np
import math
import matplotlib.pyplot as plt

def softmax(x):
    e = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

def scaled_dot_product_attention(Q, K, V):
    d_k = K.shape[-1]
    scores = np.dot(Q, np.transpose(K)) / math.sqrt(d_k)
    weights = softmax(scores)
    output = np.dot(weights, V)
    return output, weights

# Dane bazowe
K = np.array([[1., 1.], [0., 1.]])
V = np.array([[2., 0.], [0., 2.]])
Q = np.array([[1., 0.], [0.5, 1.0]])  # dwa zapytania dla dwóch głów

d_model = 2   # wymiar wejściowy
d_head = 2    # wymiar każdej głowy

# Głowa 1
Wq1 = np.array([[1., 0.], [0., 1.]])
Wk1 = np.array([[1., 0.], [0., 1.]])
Wv1 = np.array([[1., 0.], [0., 1.]])

# Głowa 2
Wq2 = np.array([[0.5, 0.2], [0.1, 0.7]])
```

```
Wk2 = np.array([[0.8, 0.1], [0.3, 0.9]])
Wv2 = np.array([[0.6, 0.4], [0.2, 0.8]])

# Połączenie - polecam do pętli będzie tak wygodniej :)
Wq_heads = [Wq1, Wq2]
Wk_heads = [Wk1, Wk2]
Wv_heads = [Wv1, Wv2]

head_outputs = []
head_weights = []

for i in range(2):
    Qi = np.dot(Q, Wq_heads[i])
    Ki = np.dot(K, Wk_heads[i])
    Vi = np.dot(V, Wv_heads[i])

    out_i, w_i = scaled_dot_product_attention(Qi, Ki, Vi)
    head_outputs.append(out_i)
    head_weights.append(w_i)

    print("Głowa", i + 1)
    print("Wagi uwagi:\n", np.round(w_i, 3))
    print("Output:\n", np.round(out_i, 3), "\n")

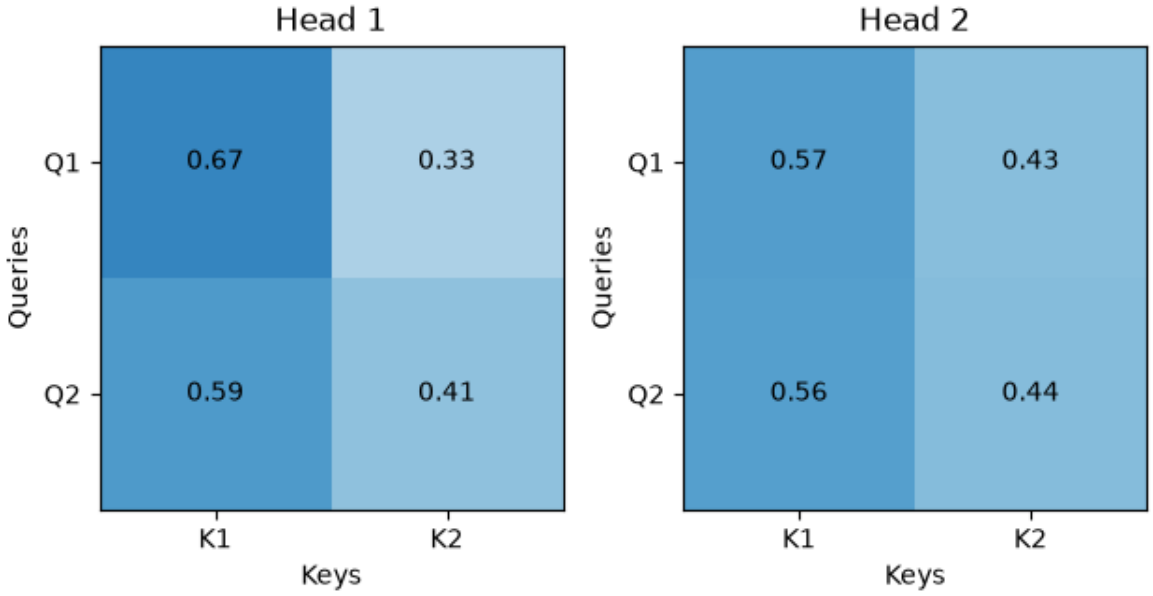
final_output = np.concatenate(head_outputs, axis=-1)
print("Końcowy wektor (konkatenacja):\n", np.round(final_output, 3))

fig, axes = plt.subplots(1, 2)
for i in range(2):
    w = head_weights[i]
    axes[i].imshow(w, cmap="Blues", vmin=0, vmax=1)
    axes[i].set_title("Head " + str(i + 1))
    axes[i].set_xlabel("Keys")
    axes[i].set_ylabel("Queries")
    axes[i].set_xticks([0, 1])
    axes[i].set_xticklabels(["K1", "K2"])
    axes[i].set_yticks([0, 1])
    axes[i].set_yticklabels(["Q1", "Q2"])
    for r in range(2):
        for c in range(2):
            axes[i].text(c, r, f"{w[r, c]:.2f}", ha="center", va="center"
plt.tight_layout()
plt.show()
```

```
Głowa 1
Wagi uwagi:
 [[0.67  0.33 ]
 [0.587 0.413]]
Output:
 [[1.34  0.66 ]
 [1.175 0.825]]

Głowa 2
Wagi uwagi:
 [[0.574 0.426]
 [0.563 0.437]]
Output:
 [[0.859 1.141]
 [0.851 1.149]]

Końcowy wektor (konkatenacja):
 [[1.34  0.66  0.859 1.141]
 [1.175 0.825 0.851 1.149]]
```




In [ ]: